情報科学演習 D

課題 3 レポート

【担当教員】　桝井 晃基，Olivier Nourry　教員

【提出者】　向井 大貴　(09B23081)
ソフトウェア科学コース・3 年
u230984j@ecs.osaka-u.ac.jp

【提出日】　2025 年 12 月 12 日

1　システムの仕様

`Checker.run(String)` に意味解析器を実装する．第一引数に与えられた ts ファイルを読み込み, 意味的な誤りがあるかを判定する．問題がなかった場合は'OK' という文字列を返し, 意味的に正しくない場合は，”Semantic error: line” という文字列とともに，最初の誤りを見つけた行の番号を返す. なお, 入力ファイルが見つからなかった場合は,”File not found” という文字列を返す．確認する項目は以下のとおりである．

- 現在のスコープに変数が存在するか
- 変数が重複して宣言されていないか
- 副プログラムが重複して宣言されていないか
- 値の型は正しいか (演算, 代入, 手続き呼び出し, 条件文, 添え字)
- 手続き呼び出し時の引数があっているか

2　課題達成の方針と設計

2.1　方針

意味解析器では, パーサで構築した AST を走査し, 変数のスコープや型などの意味的なエラーを検出する．

2.2　設計

パーサで構築した AST は Visitor パターンを採用しており, 木に影響を与えることなく全体を走査することができる．Visitor パターンで走査する際, 上位のノードから下位のノードの accept メソッドを順に呼び出していくため, 下位のノードで処理した結果を上位ノードに渡すことができる．これを利用して, 下位ノードから上位ノードに型情報を受け渡していくことで, 型のチェックを実現する．

また, 変数や副プログラムのスコープ管理にはシンボルテーブルを用いる．シンボルテーブルは, 現在のスコープで宣言されている変数や副プログラムの情報を保持し, 新たに変数や副プログラムが宣言された際に重複をチェックしたり, 変数が参照された際に存在を確認したりするために用いる．

3　実装

3.1　全体の概要

意味解析器の実装について, 処理の流れを簡単に説明する．まず, 入力ファイルを読み込み, `Token` クラスのリストとして字句解析結果を取得する．トークンファイルの読み込み及びクラスなどはこれまでの課題で実装済みであり, 詳細はレポート 1 を参照のこと．

次に, 取得したトークンリストをもとに構文解析を行う．構文解析についても, これまでの課題で実装済みであり, 詳細はレポート 2 を参照のこと．構文解析の結果,AST を得ることができる．

意味解析は既に設計方針に示したとおり,AST の Visitor パターンを活用して行う．

3.2　実装の詳細

Visitor インターフェースの実装

課題 2 で実装しておいた AST 用の Visitor のインターフェースについて説明する．`IAstVisitor` はジェネリクスを採用しており, 任意の型 T を設定できる．そして, `accept` メソッドは T を返すようにしているため, これを実装した Visitor の各ノードに対応する visit メソッドは何らかの型を返すことになる．今回は, これを利用して型情報を伝える．

```
public interface IAstVisitor<T> {
    // プログラムノードに対応する visit メソッド
    T visit(ProgramNode node);
    ...
}
```

型情報の実装

今回の pascal サブセット言語には, プリミティブ型と配列型が存在する. そのため, 基底クラス Type を用意し, これを継承する `PrimitiveType` と `ArrayType` を実装することでこれを実現している. なお,PrimitiveType は初期化時に型名の文字列をとるようになっており,Type 型のスタティック変数として Int や Boolean の PrimitiveType を宣言している. 仮に,PrimitiveType を継承する IntType などを実装した場合,Int 型を表現したいだけなのに IntType 型のインスタンスが複数宣言されることになりうる. これを防ぐためにはシングルトンパターンを採用するが, 今回は別々の型を用意する理由もなかったため, このような実装とした.

```
public abstract class Type {
    public static final Type INTEGER = new PrimitiveType("integer");
    public static final Type CHAR = new PrimitiveType("char");
    public static final Type BOOLEAN = new PrimitiveType("boolean");
}
```

Visitor の実装

意味解析を行うため,AST の各ノードを訪問する Visitor クラスを実装した. `SemanticCheckVisitor` クラスは, `IAstVisitor` インターフェースを実装することでビジターとしての機能を得る.

Visitor では, 上位のノードから下位ノードの `accept` メソッドを呼び出すことで連鎖的にノードを走査する. 例えば, 代入文ノード (AssignmentNode) に対応する visit メソッドでは, このノードが持つ左辺 (変数) ノードと右辺 (式) ノードの accept メソッドを呼び出す.

型情報は,visit メソッドの戻り値で伝える. ただし, 型情報が存在するのは式だけであり, それ以外のノードは null を返す. 型情報は, 変数や定数, 演算子ノードなど, 式を表すノードの visit メソッドで生成される. 例えば, 代入文ノードの visit メソッドでは, 右辺ノードの accept メソッドの戻り値として型情報を受け取ることができる. 変数系ノードの visit メソッドでは, シンボルテーブルから変数の型情報を取得し, それを返す.

```
@Override
public Void visit(AssignmentNode node) {
    Type expr = (Type) node.expr.accept(this); // 右辺
    Type var = (Type) node.var.accept(this); // 左辺
    if (!expr.equals(var)) {
        throw new SemanticException("...");
    }
    return null;
}
```

シンボルテーブルの実装

シンボルテーブルは, `ArrayDeque` に `Scope` オブジェクトを格納することで実現している. Scope オブジェクトは, 一意な ID を持ち, 変数ををを String 型の名前をキー,Type 型の情報を値とするマップを持つ. 新たにスコープが開始された場合, 新しい Scope オブジェクトを生成し, スタックにプッシュする. スコープが終了した場合, スタックのトッ

プの Scope オブジェクトをポップする. 変数の宣言時には, スタックのトップの Scope オブジェクトに変数情報を追加し, 変数の参照時には, スタックのトップから順に Scope オブジェクトを探索し, 変数情報を取得する.

副プログラムの宣言や呼び出しについても, 同様にシンボルテーブルで管理している. ただし, 副プログラムはスコープとは独立させグローバルなものとして扱うため,Scope オブジェクトとは別にマップで管理している. このマップは,String 型の名前をキー,SubProgrDeclInfo 型の情報を値とする. SubProgrDeclInfo 型は, 副プログラムの引数の型情報を保持するクラスである.

副プログラム呼び出しのチェックの実装

副プログラム呼び出しノード (CallNode) に対応する visit メソッドでは, まずシンボルテーブルから呼び出されている副プログラムの情報を取得する. 次に, 呼び出し時の引数リストと副プログラム宣言時の引数リストを比較し, 数や型が一致しているかをチェックする.

4 考察・工夫点

課題一から引き続き, タスクランナーの導入やテストの自動化などを行っている. 詳細はレポート 1 を参照のこと.

テスト駆動開発

課題 2 の反省を生かし, 課題 3 ではテスト駆動開発を実践してみた. テスト駆動開発とは, あらかじめ十分な単体テストを用意しておき, それぞれをパスできるように, 順に実装を進めていく方式である. あらかじめ用意されているテストでは不十分であると考えたため, 表に示したテストを追加した. なお, 独自テストの追加方法ついてはレポート 1 に記載している.

パスカルファイル名	概要	チェック内容
semerr01.pas	i=j=k (全て int 型)	型エラー検出
semerr02.pas	引数の数が合わない手続き呼び出し	引数の数エラー検出
semerr03.pas	引数の型が合わない手続き呼び出し	引数の型エラー検出

表 1: 追加したテストケース

今後を見据えた実装

意味解析器の実装にあたっては, コンパイラの実装を見据えた設計を行った.

型情報の AST への保存 意味解析器では,AST の各ノードを走査し, 型チェックを行う. コンパイラでは, 変数や式の型情報が必要となる場面がある. 例えば, 変数や文字などを出力する際には, 値がどの型であるかを知る必要がある. そのため, 意味解析器の実装にあたっては,AST の各ノードに型情報を保存する仕組みを導入した. 具体的には, 式系ノードの基底クラスである `Expr` に, 型情報を保持するフィールドを追加した. また, 変数の宣言を扱うノードである `VarDeclNode` にも, 変数の型情報を保持するフィールドを追加した.

変数名のユニーク化 コンパイラでは, 同じ名前の変数が異なるスコープで宣言されることがある. 例えば, グローバル変数とローカル変数が同じ名前で宣言されることがある. 変数名が重複していると, コンパイラにおいて予期しないバグを引き起こす可能性がある. そのため, 意味解析器の実装にあたっては, 変数名のユニーク化を行う仕組みを導入した. 具体的には, 元の変数名を name とした場合, スコープ ID を付与して name@scopeID のように変数名を変更することで, ユニーク化を実現した.

5 感想

コンパイラへ向けての第一歩として意味解析器を実装することができ, 非常に有意義な経験となった. Visitor パターンを用いた AST の走査などデザインパターンをうまく活用できたと考える. 課題 1 から 3 までを通して, コンパイラに向けての準備を終えたため, 次回以降のコード生成などの実装に向けて, さらに理解を深めていきたい.